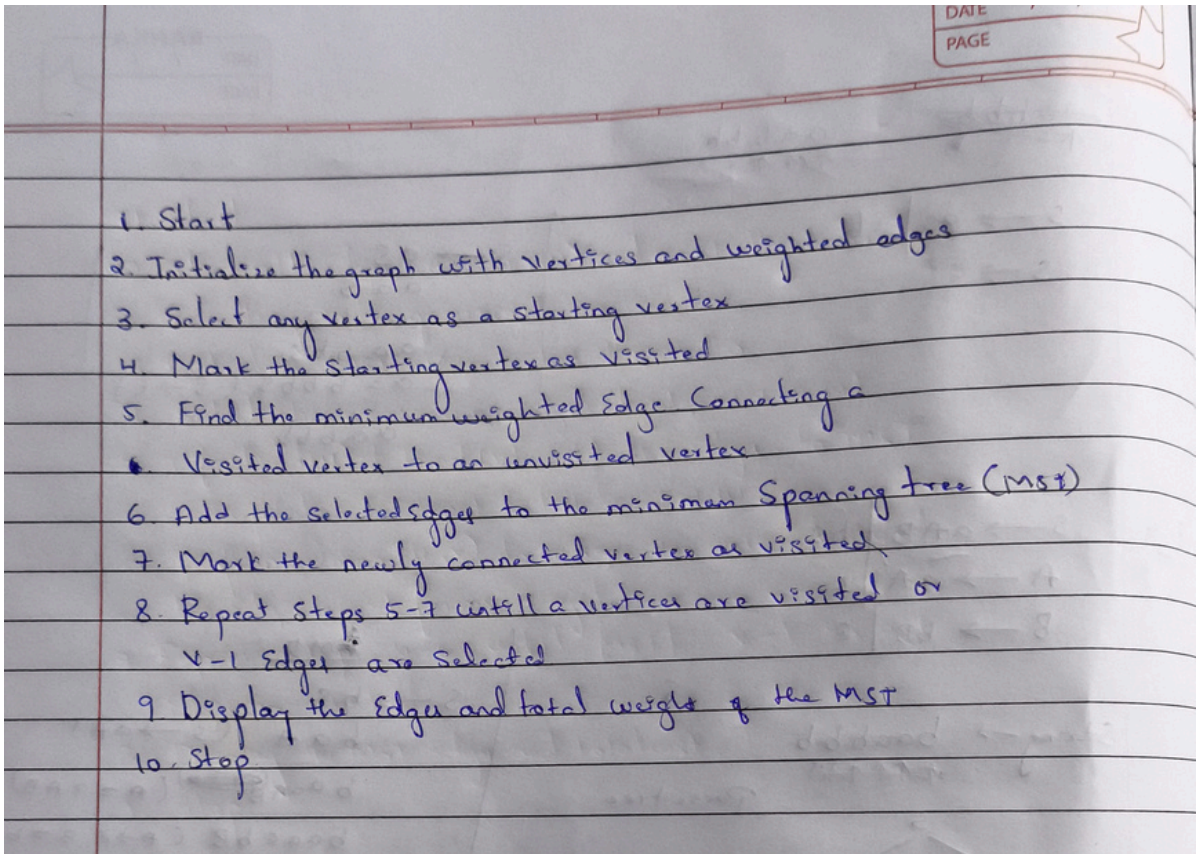
Practical: 9

Aim: Implement prim's algorithm.

Algorithm:



program code:

```
#include <iostream>
#include <stack>
using namespace std;

#define INF 9999
#define MAX 10

int main()
{
   int n;
   int graph[MAX][MAX];

   cout << "Enter number of vertices: ";
   cin >> n;

   cout << "Enter the adjacency matrix:\n";
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         cin >> graph[i][j];

         // 0 means no edge
         if (graph[i][j] == 0)
            graph[i][j] = INF;
      }
   }

   int visited[MAX] = {0};
   stack<int> s;

   int totalCost = 0;
   int edges = 0;

   // Start from vertex 0
   visited[0] = 1;
   s.push(0);

   cout << "\nEdges in Minimum Spanning Tree:\n";

   while (!s.empty() && edges < n - 1)
```

```
    {
        int minWeight = INF;
        int u = -1;
        int v = -1;

        //Checkallverticescurrently reachable
        stack<int> temp = s;

        while (!temp.empty())
        {
            intcurrent=temp.top();
            temp.pop();

            for(intj=0;j<n;j++)
            {
                if(!visited[j]&&graph[current][j] < minWeight)
                {
                    minWeight=graph[current][j];
                    u = current;
                    v=j;
                }
            }
        }

        if (u == -1)
            break;

        cout<<u<<"-"<<v
            <<"Weight:"<<minWeight << endl;

        totalCost+=minWeight;
        visited[v] = 1;
        s.push(v);

        edges++;
    }

    cout<<"\nTotalcostofMinimum Spanning Tree = "
        <<totalCost<<endl;

    return 0;
}
```

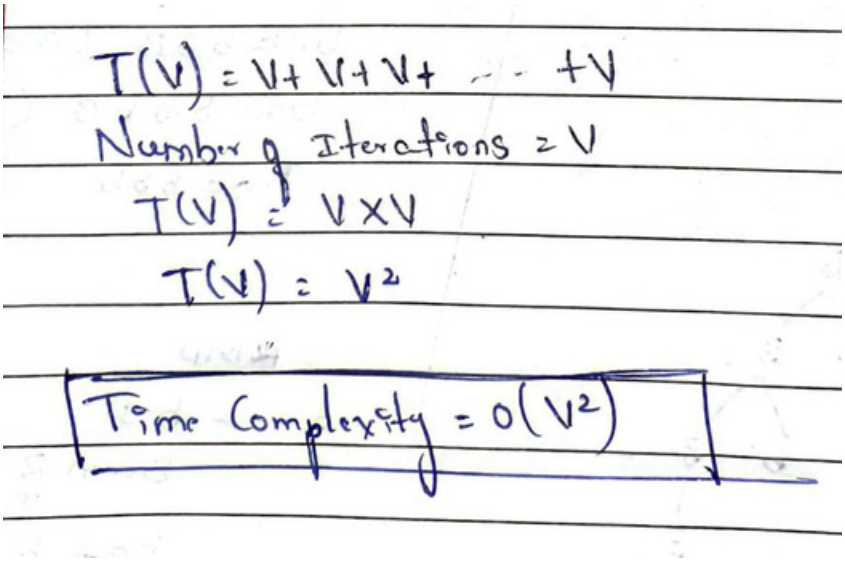
Output:

```
Enter number of vertices: 4
Enter the adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 1
6 8 1 0

Edges in Minimum Spanning Tree:
0 - 1  Weight: 2
1 - 2  Weight: 3
2 - 3  Weight: 1

Total cost of Minimum Spanning Tree = 6
```

Time complexity:



Conclusion: Thus, Prim's algorithmwas implemented successfully using a stack in C++ to find the Minimum Spanning Tree (MST) ofaweighted graph with minimum total cost.